

RAPPORT DE STAGE

Bachelor SIN/SISR — EPSI Nantes

Thanina Kana

20 avril – 1er juin 2026

Durée : 6 semaines

Entreprise d'accueil
Nantes Université

Tuteur de stage
Mohammed El Gibari

Établissement
EPSI Nantes

Remerciements

Je tiens tout d'abord à remercier M. Mohammed El Gibari, mon tuteur de stage, maître de conférences HDR à Nantes université, pour son accueil, sa disponibilité et les conseils qu'il m'a donnés tout au long de ce stage. Sa pédagogie et son soutien technique ont été précieux pour ma montée en compétences sur les systèmes embarqués.

Je remercie également Mme. Tatamo Rombeloston pour l'environnement de travail stimulant et enrichissant qu'elle m'a offert. Et un grand merci aux étudiants de master avec qui j'ai travaillée qui n'ont jamais hésité à répondre à mes questions.

Enfin, je remercie l'école EPSI Nantes pour l'accompagnement administratif et pédagogique tout au long de cette année.



Sommaire

I. Introduction

1. Présentation du candidat
2. Présentation de l'entreprise d'accueil
3. Contenu technique du stage

II. Missions réalisées

1. Prise en main de l'ESP32
2. Capteur KY-032 — Détecteur de proximité
3. Capteur KY-026 — Détecteur de flamme
4. Capteur KY-015 — Température et humidité
5. Autres capteurs
6. Connexion Wi-Fi avec l'ESP32

III. Projet final

1. Présentation du projet
2. Branchement et intégration des capteurs
3. Développement du code ESP32
4. Mise en place de la base de données (MySQL / phpMyAdmin)
5. Développement de l'interface web (PHP, HTML, CSS)
6. Résultat final et accès avec ngrok
7. Boîtier en Impression 3D

IV. Axes d'amélioration

V. Conclusion

VI. Bibliographie

VII. Annexes

I. Introduction

1. Présentation du candidat

Je m'appelle Thanina Kana et je suis étudiante en première année de Bachelor SIN (Systèmes informatiques et numériques) à l'EPSI de Nantes, Passionnée par les infrastructures, l'administration système et la cybersécurité. J'ai souhaité effectuer ce stage dans un laboratoire de recherche afin de découvrir le monde des systèmes embarqués et de la transmission sans fil.

Ce stage d'une durée de 6 semaines, du 20 avril au 1^{er} juin 2026, s'est déroulé au sein de l'IETR (Institut d'Électronique et des Technologies du numéRique) à Nantes Université, sous la supervision de M. Mohammed El Gibari.

2. Présentation de l'entreprise d'accueil

Nantes Université est un établissement public d'enseignement supérieur et de recherche situé à Nantes, en Pays de la Loire. Héritière d'une longue tradition académique, elle regroupe aujourd'hui plus de 40 000 étudiants et 3 000 enseignants-chercheurs répartis sur plusieurs campus et composantes couvrant un large spectre de disciplines : sciences et technologies, lettres et langues, droit et sciences politiques, médecine et santé, économie et gestion.

L'université s'inscrit dans un réseau de recherche national et international, collaborant avec de nombreux organismes tels que le CNRS, l'INSERM ou encore l'INRAE. Elle dispose de nombreuses unités de formation et de recherche, parmi lesquelles l'UFR Sciences et Techniques, composante scientifique majeure de l'établissement, implantée sur le campus situé au 2 rue de la Houssinière à Nantes. Cette UFR accueille des formations allant de la licence au doctorat dans des domaines tels que la physique, l'électronique, l'informatique, les mathématiques et les sciences de l'ingénieur. Elle dispose également de nombreux laboratoires et plateformes expérimentales permettant la conduite de travaux de recherche avancés.

Le bâtiment 14, au sein duquel s'est déroulé mon stage, est dédié aux travaux pratiques de physique, d'électronique et d'informatique. Il accueille à la fois des étudiants en formation et des enseignants-chercheurs menant leurs activités de recherche dans un environnement technique et scientifique stimulant.



C'est dans ce cadre que j'ai eu la chance d'être accueillie par mon tuteur M. Mohammed El Gibari, au Département de Physique de Nantes Université, spécialisé dans les capteurs et les systèmes embarqués connectés.

3. Contenu technique du stage

Au cours de ce stage, j'ai travaillé sur la thématique ESP32 et la transmission sans fil. Les missions ont été organisées en quatre phases principales selon le planning établi :

- Introduction aux microcontrôleurs ESP32, installation de l'Arduino IDE, premiers programmes
- Capteurs simples, acquisition de données, communication Wi-Fi/Bluetooth
- Développement d'une interface web pour la visualisation graphique
- Mini projet intégrant capteur, transmission et interface.

II. Missions réalisées

1. Prise en main de l'ESP32

a. Présentation

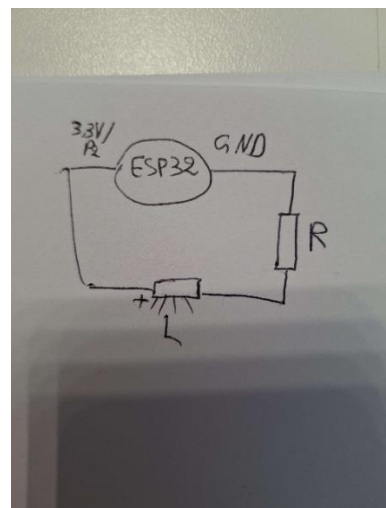
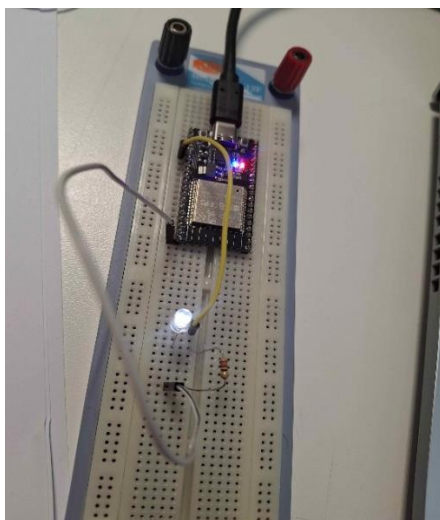
L'ESP32 est un microcontrôleur double cœur développé par Espressif Systems. Il intègre le Wi-Fi et le Bluetooth, ce qui en fait une plateforme très adaptée aux projets IoT. Il dispose de 38 broches, des GPIO des ADC des GND et une broche 3V3 ainsi qu'une 5V.

Ma première mission a consisté à allumer et éteindre une LED connectée à une broche GPIO, puis à lui faire afficher des signaux en code morse (SOS et OK). Cet exercice permet de maîtriser les fonctions de base d'Arduino.



b. Fonctionnement / Branchement

Le branchement est simple, la LED est connectée entre une broche GPIO et la masse (GND), avec une résistance en série pour limiter le courant.





c. Code utilisé

Deux programmes distincts ont été réalisés :

Code 1 — LED simple (clignotement) :

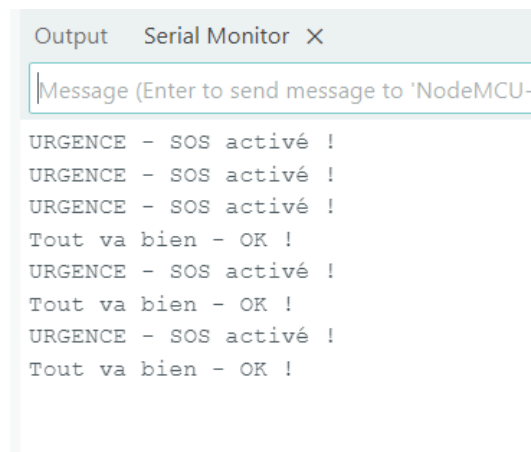
```
void setup() {  
  pinMode(2, OUTPUT); // broche 2 =  
  sortie  
}  
void loop() {  
  digitalWrite(2, HIGH); // LED allumée  
  delay(500);  
  digitalWrite(2, LOW); // LED éteinte  
  delay(500);  
}
```

Code 2 — SOS / OK en morse :

```
bool urgence = false;  
  
void loop() {  
  if(digitalRead(0) == LOW) {  
    urgence = !urgence;  
    delay(500);  
  }  
  if(urgence) {  
    sos();  
  } else {  
    ok();  
  }  
}
```

d. Résultat obtenu

La LED clignote correctement a intervals réguliers pour le premier programme. Pour le second, le signal morse est visible a l'œil nu et correspond bien aux durées définies dans le code, et un message s'affiche dans le serial monitor :



```

Output  Serial Monitor X
Message (Enter to send message to 'NodeMCU-
URGENCE - SOS activé !
URGENCE - SOS activé !
URGENCE - SOS activé !
Tout va bien - OK !
URGENCE - SOS activé !
Tout va bien - OK !
URGENCE - SOS activé !
Tout va bien - OK !
  
```

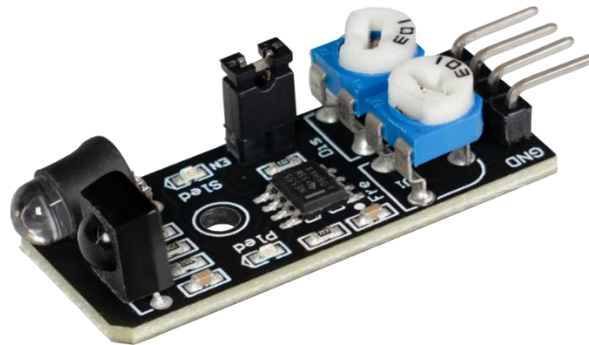
e. Problèmes rencontrés et solutions

- Le port COM ne s'affichait pas dans l'Arduino IDE, nécessitant l'installation manuelle du driver CP210x
- Première tentative d'upload échouée avec "No serial data received " le bouton BOOT devait être maintenu pendant l'upload
- La logique inversée du bouton BOOT (HIGH au repos, LOW à l'appui) difficile a comprendre au départ
- Durées des points et traits morse à calibrer plusieurs fois pour être lisibles

2. Capteur KY-032 (Détecteur infrarouge d'obstacle)

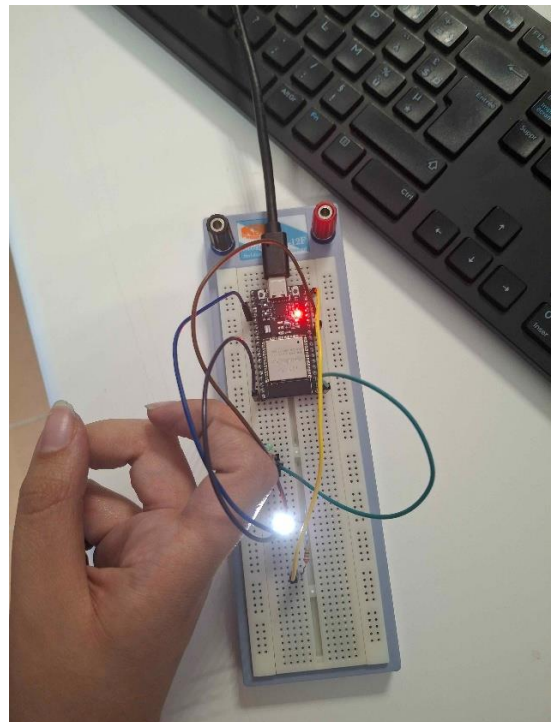
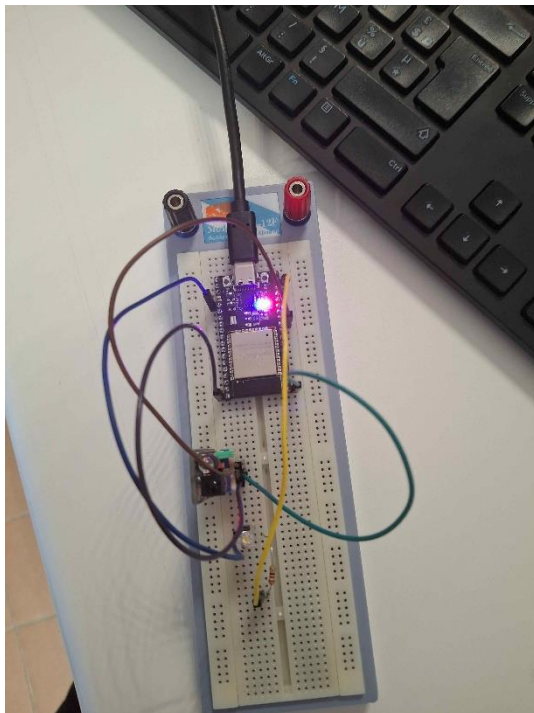
a. Présentation

Le ky-032 est un module capteur infrarouge qui détecte la présence d'un obstacle à courte distance. Il émet un faisceau infrarouge et analyse la réflexion : si un objet se trouve dans sa portée (réglable via un potentiomètre de quelques centimètres à une trentaine de centimètres), la sortie numérique passe a l'état low. Il dispose de trois broches : VCC, GND et OUT.



b. Fonctionnement / Branchement

Le KY-032 se branche directement sur l'ESP32 : VCC sur 3.3V, GND sur GND et OUT sur une broche GPIO en entrée numérique.



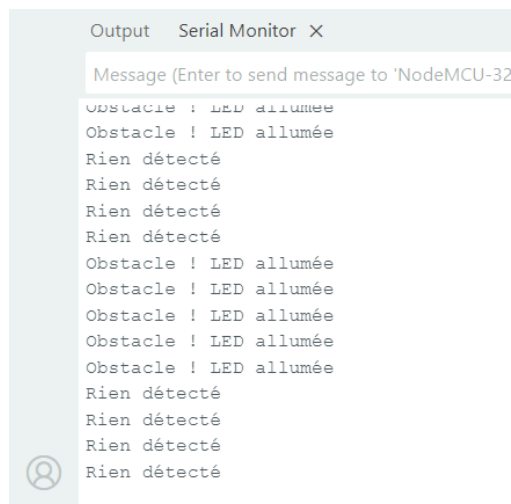
c. Code utilisé

```
void loop() {
  if (digitalRead(27) == HIGH) {
    digitalWrite(2, HIGH);
    Serial.println("Rien détecté");
  } else {
    digitalWrite(2, LOW);
  }
}
```

```
Serial.println("Obstacle ! LED allumée");
}
delay(100);
}
```

d. Résultat obtenu

Lorsqu'un obstacle est placé devant le capteur, la LED s'allume et le message « Obstacle ! LED allumée » s'affiche dans le moniteur série. Le retrait de l'obstacle éteint la LED et affiche le message « Rien détecté » dans le moniteur série.



e. Problèmes rencontrés et solutions

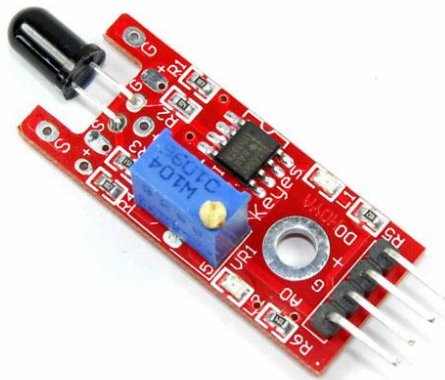
- Le câblage signal branché sur la mauvaise broche GPIO au départ, ce qui empêchait toute détection et nécessitait de vérifier le tableau de câblage
- Au démarrage le capteur ne répondait pas après vérification c'était le délai dans le loop qui était trop court pour que le capteur ait le temps de traiter le signal entre deux lectures
- Le capteur détectait des obstacles de manière aléatoire même en l'absence de tout objet, il a fallu ajuster manuellement le potentiomètre intégré au module pour calibrer la sensibilité à la bonne distance de détection
- Premier branchement incorrect, VCC branche à 5V au lieu de 3.3V ce qui risquait d'endommager le module

- Le moniteur série n'affichait rien au démarrage, il fallait s'assurer que le baud rate du moniteur série dans l'Arduino IDE correspondait bien à celui défini dans le `Serial.begin()` du code

3. Capteur KY-026 — Détecteur de flamme

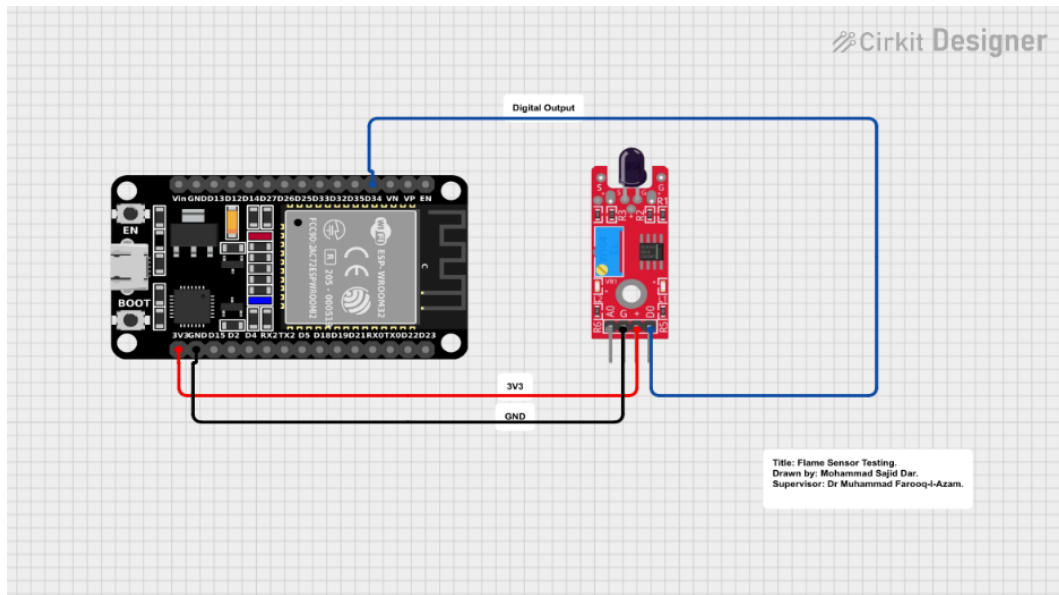
a. Présentation

Le KY-026 est un module capteur de flamme fonctionnant dans l'infrarouge. Il est sensible au rayonnement infrarouge émis par une flamme nue (bougie, briquet). Il dispose d'une sortie numérique DO et une sortie analogique AO permettant de détecter la présence d'une flamme et d'en estimer l'intensité.



b. Fonctionnement / Branchement

Le KY-026 se connecte à l'ESP32 via quatre broches : VCC (3.3 V), GND, DO (sortie numérique sur GPIO) et optionnellement AO (sortie analogique sur une broche ADC), je n'ai pas utilisé cette dernière. La sensibilité est ajustable via le potentiomètre intégré au module.



c. Code utilisé

```
void loop() {
  int flamme = digitalRead(14);
  if (flamme == LOW) {
    Serial.println("Flamme détectée !");
  } else {
    Serial.println("Pas de flamme.");
  } delay(300);
}
```

d. Résultat obtenu

Le capteur détecte correctement la présence d'une flamme et la LED s'allume en conséquence. La valeur analogique varie en fonction de l'intensité et de la distance de la flamme.



```
Output  Serial Monitor  X
Message (Enter to send message to
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
Pas de flamme.
```

e. Problèmes rencontrés et solutions

- Le capteur ne détectait aucune flamme lors des premiers tests, le potentiomètre de sensibilité n'était pas correctement réglé, un ajustement manuel a été nécessaire
- Difficulté à tester sans source de flamme réelle disponible, le test a finalement été réalisé avec un briquet
- La sortie numérique DO renvoyait parfois des valeurs instables, un délai suffisant entre chaque lecture dans loop() a permis de stabiliser le signal

4. Capteur KY-015 — Température et humidité

a. Présentation

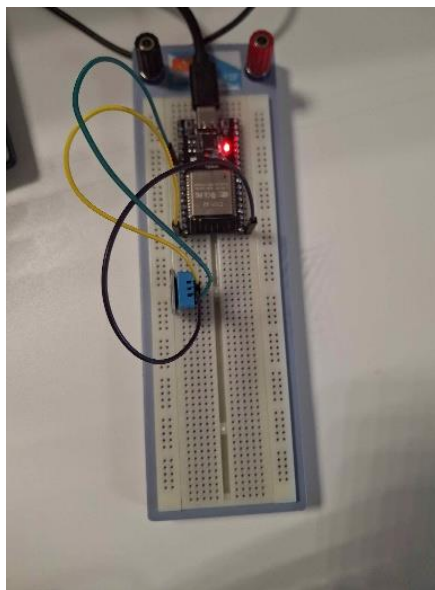
Le module KY-015 embarque un capteur DHT11 capable de mesurer la température ambiante (entre 0- 50°C avec une précision de plus au moins 2°C) ainsi que le taux d'humidité (entre 20- 90% avec une précision de plus au moins 5%).



b. Fonctionnement / Branchement

Le KY-015 se branche sur trois broches à l'ESP32 : VCC à 3.3V le GND au GND, et le DATA à une broche GPIO.

Et pour fonctionner il fallait installer la bibliothèque DHT d'Adafruit sur L'IDE



c. Code utilisé

```
#include <DHT.h>

void loop() {
  delay(2000);
  float humidite = dht.readHumidity();
  float temperature = dht.readTemperature();
```



```
if (isnan(humidite) || isnan(temperature)) {
  Serial.println("Erreur de lecture du capteur !");
  return;
}

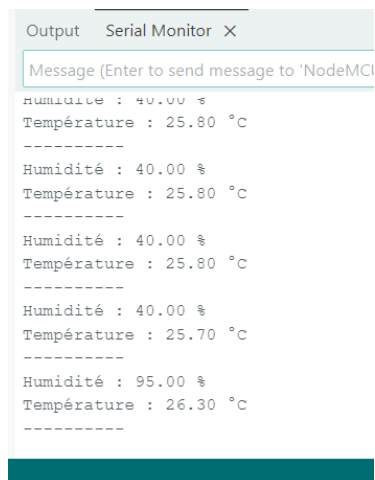
Serial.print("Température : ");
Serial.print(temperature);
Serial.println(" °C");

Serial.print("Humidité : ");
Serial.print(humidite);
Serial.println(" %");

Serial.println("-----");
}
```

d. Résultat obtenu

Les valeurs de température et d'humidité s'affichent correctement dans le moniteur série toutes les 2 secondes.



```
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU')
humidite : 40.00 %
Température : 25.80 °C
-----
Humidité : 40.00 %
Température : 25.80 °C
-----
Humidité : 40.00 %
Température : 25.80 °C
-----
Humidité : 40.00 %
Température : 25.70 °C
-----
Humidité : 95.00 %
Température : 26.30 °C
-----
```

e. Problèmes rencontrés et solutions

- Erreur de lecture fréquente au démarrage, le DHT11 nécessite un délai de 2 seconds minimums entre chaque lecture sans quoi il renvoyait une erreur

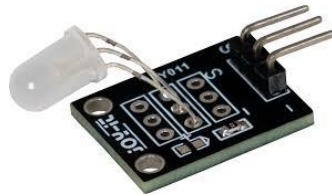
- Câblage incorrect au départ, la broche DATA branchée était branchée sur le mauvais GPIO corrigée en revérifiant le schéma de câblage

5. Autres capteurs testés

Dans le cadre de l'exploration du kit SensorKit X40, j'ai également testé plusieurs capteurs en dehors de ceux retenus pour le projet final.

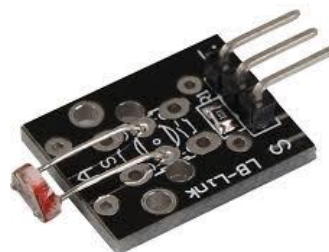
a. KY-011 (LED RGB bicolore) :

Le test de ce capteur n'a pas abouti en raison d'un problème de câblage lié à la taille réduite de la breadboard. J'ai également testé les modules similaires KY-016 et KY-029, sans résultat pour les trois. Ces capteurs ont été abandonnés.



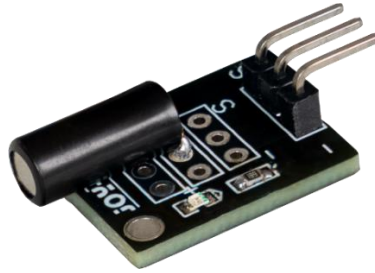
b. KY-018 (photorésistance) :

Le premier branchement a provoqué une surchauffe de l'ESP32 en raison d'un câblage incorrect. Après correction, le capteur fonctionnait correctement en renvoyant une valeur proportionnelle à la luminosité ambiante. Un second test visant à allumer une LED lorsque la luminosité était insuffisante n'a cependant pas marché.



c. KY-017 (capteur à bascule) :

Ce capteur détecte l'inclinaison ou les vibrations grâce à une bille métallique en son intérieur, renvoyant un signal numérique HIGH ou LOW selon son orientation. Ce dernier test a été réalisé avec succès.



e. KY-037 (capteur de son) :

Ce capteur détecte les sons ambiants et renvoie un signal numérique lorsque le niveau sonore dépasse un seuil réglable via potentiomètre. Ce dernier test a été réalisé avec succès.



Les capteurs KY-032, KY-026 et KY-015 ont été retenus pour le projet final car ils correspondent le mieux aux besoins d'une maison connectée : détection de présence, sécurité incendie et surveillance de l'environnement.

5. Connexion Wi-Fi avec l'ESP32

a. Présentation

L'ESP32 intègre nativement une puce Wi-Fi ce qui lui permet de se connecter à un réseau local dans un composant supplémentaire. Cette fonctionnalité est indispensable pour le projet final : c'est par le Wi-Fi que l'ESP32 envoie les données des capteurs vers le serveur PHP.

Cette étape avait pour objectif de tester et valider la connexion Wi-Fi de l'ESP32 avant de l'intégrer dans le projet final

b. Fonctionnement / Branchement

Aucun branchement n'est nécessaire pour cette étape.



c. Code utilisé

```
WiFi.disconnect(true);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED && t < 30) {
  delay(500);
  Serial.print(".");
  t++;
}

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("Connecté !");
  Serial.println(WiFi.localIP());
} else {
  Serial.println("Échec WiFi");
}
```

d. Résultat obtenu

Une fois le code téléversé, le moniteur série affiche « connecté ! » suivi de l'ip local de l'ESP32 attribuée par le routeur

```
Connexion WiFi...
.....
Connecté !
IP : 10.38.120.212
```

e. Problèmes rencontrés et solutions

- Brownout detector déclenché au début de la connexion, résolu avec la ligne `WRITE_PERI_REG`
- Connexion via partage de connexion ne fonctionnait pas au début car réseau masqué
- Echec de connexion lors des premiers essais, résolu en ajoutant `WiFi.disconnect(true)`



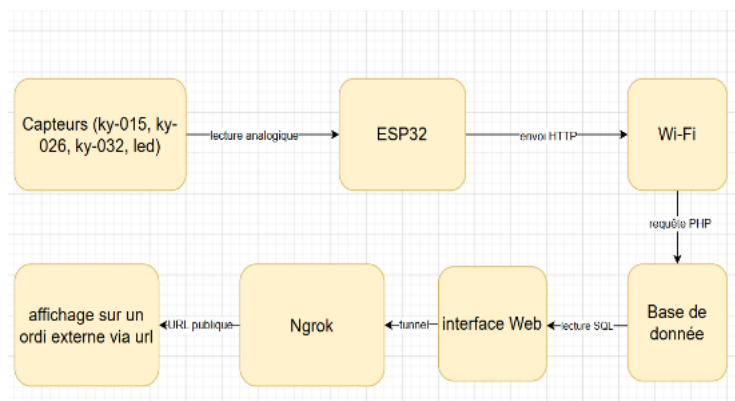
- Affichage de points dans le serial monitor sans connexion aboutie, résolu en limitant les tentatives à 30 et en affichant le statut d'erreur

III. Projet final : Maison connectée

1. Présentation et architecture du système

L'objectif du projet final est de concevoir un système de maison connectée capable de surveiller en temps réel l'environnement d'un logement. L'idée est de regrouper les différents capteurs sur un seul microcontrôleur afin de centraliser la surveillance, le tout accessible depuis n'importe quel appareil connecté.

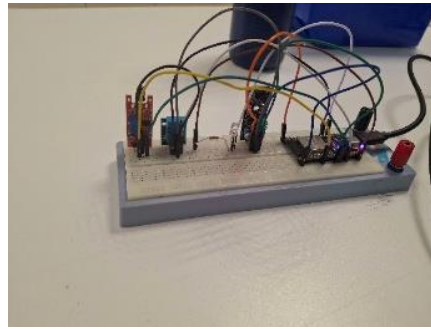
Ce système suit une architecture en couches successives, l'ESP32 lit les données des capteurs en continu et les transmet via protocole HTTP au serveur local. Ce serveur, hébergée sous XAMPP, reçoit les données en PHP et les insert dans une base de données MySQL. Une seconde page PHP interroge ma base de données et renvoie les informations en format JSON a l'interface web qui les récupère toutes les deux secondes via AJAX et met a jour l'affichage sans avoir à rafraichir la page. Enfin l'outil ngrok permet d'exposer le serveur local sur internet rendant l'interface accessible depuis l'extérieur.



2. Branchement et intégration des capteurs

Une fois les capteurs testés individuellement, ils ont été intégrés simultanément sur la breadboard afin de constituer le montage final.

Le KY-032 est connecté sur le GPIO 27, le KY-026 sur le GPIO 14, et le KY-015 sur le GPIO 4. La led utilisée avec le KY-032 est sur le GPIO 2.



3. Développement du code ESP32

a. Lecture des capteurs

L'ESP32 lit simultanément les trois capteurs à chaque itération. Le KY-032 et le KY-026 sont lus via `digitalRead()` sur leurs broches GPIO respectives. Les valeurs seront converties en 0 ou en 1 pour l'insertion en base de données. Le KY-015 est lu via la bibliothèque DHT qui gère le protocole one-wire et retourne la température en degrés Celsius et l'humidité en pourcentage.

```
int obstacle = digitalRead(OBSTACLE_PIN);
int flamme  = digitalRead(FLAME_PIN);
float humidite  = dht.readHumidity();
float temperature = dht.readTemperature();
```

b. Envoi des données vers la base de données

L'envoi des données se fait via une requête HTTP GET construite dynamiquement avec les valeurs lues. Pour éviter les doublons en base de données l'envoi n'est déclenché que si au moins une valeur a changé par rapport à la lecture précédente.

```
if (changement && WiFi.status() == WL_CONNECTED) {
  HTTPClient http;
  String url = String(serverURL) +
    "?temperature=" + String(temperature) +
    "&humidity="  + String(humidite) +
    "&obstacle="  + String(obs_val) +
```



```
"&flamme=" + String(flamme_val);  
http.begin(url);  
int httpCode = http.GET();  
http.end();  
}
```

4. Mise en place de la base de données (MySQL / phpMyAdmin)

La base de données a été créée sous MySQL via l'interface PhpMyAdmin accessible depuis XAMPP elle porte le nom maison_ESP32 et contient une seule table nommée données, conçue pour stocker l'ensemble des relevés envoyés par l'ESP32 la table donnée est composée des colonnes suivantes :

- Id : entier auto-incrémenté, clé primaire, identifiant unique de chaque enregistrement
- Temperature : nombre décimal (FLOAT) stockant la température en degrés Celsius
- Humidité : nombre décimal (FLOAT) stockant le taux d'humidité en pourcentage
- Obstacle : entier valant 0 ou 1 indiquant la présence ou l'absence d'un obstacle
- Flamme : entier valant 0 ou 1 indiquant la détection d'une flamme
- timestamp : date et heure de l'enregistrement de la valeur



				id	temperature	humidite	obstacle	flamme	timestamp
☐	✎ Éditer	📋 Copier	🗑 Supprimer	1	22.3	48	0	0	2026-05-05 10:24:59
☐	✎ Éditer	📋 Copier	🗑 Supprimer	2	22.3	48	1	0	2026-05-05 10:25:07
☐	✎ Éditer	📋 Copier	🗑 Supprimer	3	22.3	48	0	0	2026-05-05 10:25:10
☐	✎ Éditer	📋 Copier	🗑 Supprimer	4	22.2	48	0	0	2026-05-05 10:25:11
☐	✎ Éditer	📋 Copier	🗑 Supprimer	5	22.3	48	0	0	2026-05-05 10:25:13
☐	✎ Éditer	📋 Copier	🗑 Supprimer	6	22.3	48	1	0	2026-05-05 10:25:20
☐	✎ Éditer	📋 Copier	🗑 Supprimer	7	22.3	48	0	0	2026-05-05 10:25:23

5. Développement de l'interface web (PHP, HTML, CSS)

a. Structure des fichiers

L'interface web repose sur une architecture PHP organisée en fichiers aux responsabilités séparées. Le fichier `bdd.php` centralise la connexion MySQL et est inclus via `include` dans chaque page, évitant toute duplication. Le fichier `insert.php` reçoit et valide les données envoyées par l'ESP32 via HTTP GET, en utilisant des requêtes préparées pour prévenir l'injection SQL. Le fichier `data.php` interroge la base de données et retourne un objet JSON structuré contenant les dernières mesures, l'historique des 20 dernières lignes, l'ensemble des présences détectées et toutes les alertes flammes. Le fichier `export_data.php` retourne quant à lui la totalité des données sans limites pour l'export Excel.

b. Mise à jour en temps réel avec AJAX

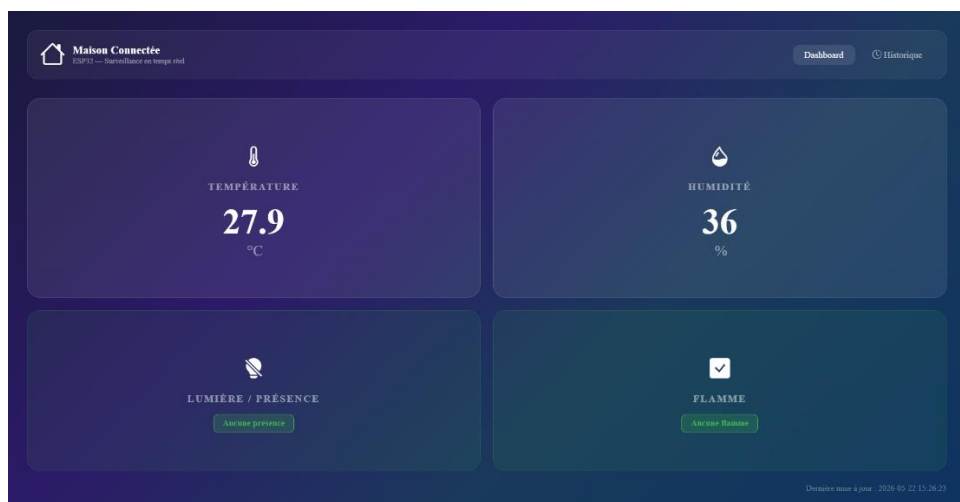
AJAX (asynchronous javascript and XML) est une technique permettant d'effectuer des requêtes HTTP depuis le navigateur sans recharger la page. Dans ce projet une fonction javascript interroge `data.php` toutes les secondes via l'API `fetch()` native, récupère les réponses JSON et mets à jour le DOM dynamiquement. Cette approche évite tout rechargement de page et donne à l'interface un comportement temps-réel.

```
function mettreAJour() {
  fetch('data.php')
  .then(response => response.json())
  .then(data => {
```

```
document.getElementById('temperature').textContent =
data.temperature;
document.getElementById('humidite').textContent = data.humidite;
document.getElementById('timestamp').textContent = data.timestamp;
});
}
mettreAJour();
setInterval(mettreAJour, 1000);
```

c. Affichage des données

Le dashboard est composé de quatre cartes disposées en grilles 2x2, chacune dédiée à un capteur. Les icônes proviennent de Bootstrap et sont mises à jour dynamiquement via JavaScript selon l'état des capteurs, sans rechargement de page. L'interface est entièrement responsive grâce à un menu burger pour mobile, implémenté en JavaScript pur sans bibliothèques externes.

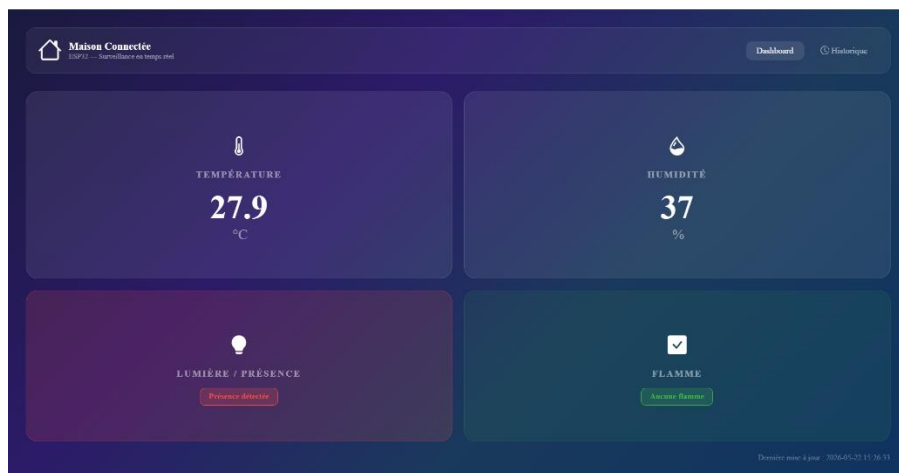


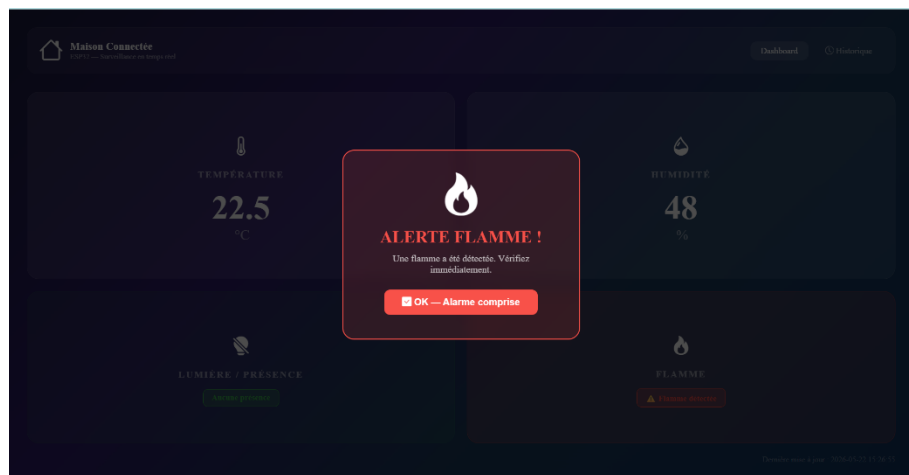
d. Alertes visuelles et sonores

Le système d'alerte repose sur deux mécanismes complémentaires. D'une part les cartes du dashboard changent dynamiquement de couleur en se basant sur l'état renvoyé par data.php (rouge en cas de danger vert en situation normale). D'autre part en cas de détection de flammes un pop-up bloquant s'affiche et une alerte sonore est déclenchée via le web audio API. Une API intégrée dans les navigateurs modernes qui permet de générer des sons directement en javascript sans fichiers

audio externe. L'alarme génère un oscillateur à 880 hz répété toutes les 600 ms jusqu'à confirmation explicite de l'utilisateur. Garantissant qu'une alerte ne passe pas inaperçue.

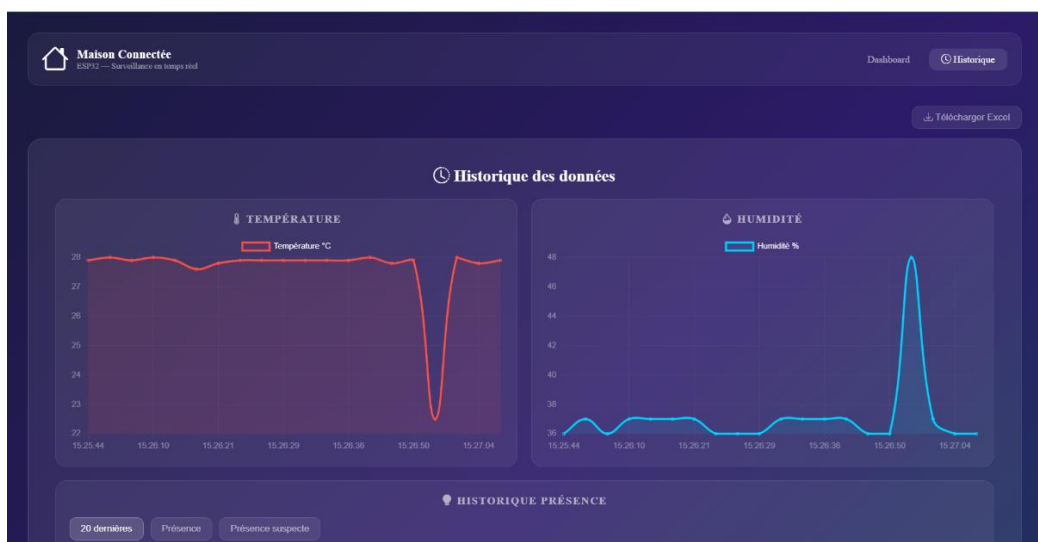
```
function bip() {
  const oscillator = audioCtx.createOscillator();
  const gainNode = audioCtx.createGain();
  oscillator.connect(gainNode);
  gainNode.connect(audioCtx.destination);
  oscillator.type = 'square';
  oscillator.frequency.setValueAtTime(880, audioCtx.currentTime);
  gainNode.gain.setValueAtTime(0.3, audioCtx.currentTime);
  gainNode.gain.exponentialRampToValueAtTime(0.001,
    audioCtx.currentTime + 0.4);
  oscillator.start(audioCtx.currentTime);
  oscillator.stop(audioCtx.currentTime + 0.4);
}
alarmeInterval = setInterval(bip, 600);
```





e. Graphiques température et humidité

La page historique intègre deux graphiques en courbes générés par chart.js, une bibliothèque javascript de visualisation de données la première affiche de l'évaluation de la température en rouge et le second affiche l'humidité en bleu. Les graphiques sont reconstruits toutes les trois secondes à partir des données Ajax. Les données précédentes sont détruites via `chartTemp.destroy()` avant chaque reconstruction pour éviter les fuites mémoires .



f. Historique et filtres

La page historique propose un système de filtrage avancé pour l'historique des présences, trois filtres sont disponibles : les « 20 derniers » affiche les 20 dernières mesures toutes catégories confondues, « présence » affiche uniquement la

détection de présence en dehors des heures nocturnes et « présence suspecte » isole les détections survenues entre 00h00-06h00 du matin, considéré comme suspecte. Cette logique entièrement gérée côté javascript sans requête supplémentaire vers le serveur, les données de présence complète sont chargées une seule fois depuis data.php et filtrer dynamiquement à chaque changement de filtre.

HISTORIQUE PRÉSENCE

20 dernières Présence Présence suspecte

DATE / HEURE	STATUT
2026-05-20 01:23:00	Présence suspecte
2026-05-20 01:23:00	Présence suspecte
2026-05-19 01:23:00	Présence suspecte

ALERTE FLAMME

- 2026-05-22 15:26:55
- 2026-05-22 14:43:17
- 2026-05-22 14:33:08
- 2026-05-22 14:20:32
- 2026-05-22 11:31:21
- 2026-05-20 10:10:26
- 2026-05-20 09:44:02
- 2026-05-18 15:35:03

g. Export Excel

L'interface propose un export des données au format Excel (.xlsx) via la bibliothèque SheetJS. L'export est généré entièrement côté navigateur à partir des données récupérées depuis export_data.php qui retourne la totalité des enregistrements sans limite. Le fichier excel est organisé en feuilles par jour chaque feuille contenant trois tableaux côte à côte : Température et humidité, historique des présences et alerte flamme. Le fichier est nommé automatiquement avec la date du téléchargement.

Page 28

Le boîtier a été conçu et imprimé en PLA grâce à une imprimante Prusa. Le PLA a été choisi pour ses propriétés isolantes afin d'éviter tout risque de court-circuit avec les composants électroniques, le bois ayant été écarté pour cette même raison le fichier de modélisation a été préparé avec inkscape puis converti pour l'impression via prusaSlicer. La durée d'impression était d'environ 14 h.

Le boîtier a été dimensionné pour accueillir la breadboard avec une hauteur suffisante pour ne pas écraser les fils des capteurs, un trou a été prévu sur le côté large pour le passage du câble USB de l'ESP32. Le couvercle conçu en système coulissant et gravé du logo d'une maison ainsi que du nom du projet "*Maison Connectée ESP32*".



IV. Axes d'amélioration

Bien que le système soit pleinement fonctionnel, plusieurs pistes d'amélioration ont été identifiées au cours du développement :

a. Buzzer sonore physique : L'alerte sonore est actuellement générée via le web audio API du navigateur, qui implique que le son n'est audible que si l'interface est ouverte sur un appareil. L'ajout d'un buzzer physique directement connecté à l'ESP32 permettrait une alerte sonore indépendante de l'interface web

b. Sonnette connectée : Le capteur KY-032 pourrait être détourné pour simuler une sonnette connectée, déclenchant une notification sur l'interface web lorsqu'une présence est détectée à l'entrée

c. Passage en HTTPS local : Le serveur XAMPP tourne actuellement en HTTP. La mise en place d'un certificat SSL local permettrait de sécuriser les échanges entre l'ESP32 et le serveur, même en réseau local

d. Filtrage par dates dans l'export Excel : L'export Excel génère actuellement un fichier contenant toutes les données depuis le début, l'ajout d'un sélecteur de plage de dates permettrait à l'utilisateur de cibler une période précise

e. Synchronisation Google Sheets : L'export des données en temps réel vers Google sheets via l'API Google permettrait de consulter les données sans télécharger à nouveau Excel à chaque fois

f. Graphiques avancés : Les graphiques chart.JS affiche uniquement les 20 dernières mesures, l'intégration de filtres temporels enrichirait considérablement la page historique

V. Conclusion

Pour conclure, ce stage de six semaines à l'IETR a représenté une expérience professionnelle riche et formatrice. Il m'a permis de découvrir le monde des systèmes embarqués et de la transmission sans fil, des domaines que je ne connaissais pas avant.

Sur le plan technique, j'ai acquis des compétences en programmation embarquée, en communication sans fil et en développement d'interfaces de visualisation. Ces apprentissages complètent ma formation et enrichissent mon profil dans la perspective d'un master en cybersécurité.

Sur le plan humain, cette expérience m'a permis de travailler en autonomie dans un environnement de recherche, de développer ma rigueur et mon sens de l'organisation, et de renforcer ma motivation à évoluer dans le secteur des nouvelles technologies.



VI. Bibliographie

- Forum Arduino : Communauté d'entraide et de documentation pour le développement sur microcontrôleurs <https://forum.arduino.cc>
- Espressif Systems : Documentation officielle de l'ESP32 <https://docs.espressif.com>
- Joy-IT : Documentation technique des capteurs du kit SensorKitX40 <https://joy-it.net>
- Créa Troyes : Tutoriel ngrok rendre un serveur local accessible sur le web <https://blog.crea-troyes.fr/13784/ngrok-votre-serveur-local-localhost-sur-le-web-tutoriel/>

VII. Annexes

Annexe 1 :

Semaine 1	Prise en main ESP32, LED, code Morse SOS/OK
Semaine 2	Tests capteurs KY-032, KY-026, KY-015, KY-017, KY-037, KY-018
Semaine 3	Intégration capteurs, connexion WiFi, base de données MySQL
Semaine 4	Interface web PHP/HTML/CSS, AJAX, alertes visuelles et sonores
Semaine 5	Page historique, graphes Chart.js, export Excel, FabLab boîtier
Semaine 6	Finalisation, tests, rapport, présentation

Annexe 2 :

Lien GitHub : <https://github.com/thanina-kana/maison-connectee-ESP32>

Annexe 3 :

Hardware :

- ESP32 NodeMCU-32S
- Kit SensorKit X40 (Joy-IT)
- Breadboard RS PRO 780 points
- Câble USB
- Imprimante 3D Prusa (*Atelier 119, FabLab Nantes Université*)

Logiciels :

- Arduino IDE 2.3.8
- XAMPP (Apache + MySQL)
- phpMyAdmin
- PhpStorm
- draw.io
- ngrok



- Inkscape (*modélisation du boîtier*)
- PrusaSlicer (*découpe pour impression 3D*)